

STUDENT GRADE MANAGEMENT SYSTEM (SGMS)

Build Your Own Project - Project Report

Prepared by	Devansh Tripathi
University	Vellore Institute of Technology Bhopal
Course / Subject	Introduction to Problem Solving and Programming
Programme	B.Tech CSE (Core)

Note: The report describes the exact source file submitted with this project. No changes have been made to the original SGMS.py for this report package.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

Project repository: github.com/devansh26bce10750/SGMS

1. Introduction

The Student Grade Management System (SGMS) is a small Python console application that I built to practice basic programming concepts in a practical way. Instead of treating marks as separate calculations on paper, the program takes the information once and keeps it in a list while the program is running.

The current source accepts a roll number, student name, and marks for Python, Mathematics, and Programming. It then calculates the total and average and stores the result in a dictionary. The menu also provides options to display records, search by roll number, and find the highest scorer.

I deliberately kept the interface simple. For this project, a console menu is enough to show the Python concepts clearly without adding an unrelated graphical framework or database.

2. Problem Statement

Keeping track of student marks manually is repetitive. For each student, total and average marks have to be calculated, records have to be looked up, and the highest scorer may have to be found by comparing multiple records. A small menu-driven program can perform those routine operations in a consistent way.

The scope of this version is limited to records held in memory during one execution of the program. There is no database or permanent file storage, so the information is not retained after the program is closed.

3. Objectives

The main objectives of the project are:

- to accept and organise basic student academic information;
- to calculate total and average marks automatically;
- to provide simple display and roll-number search operations;
- to identify the student with the highest average;
- to demonstrate lists, dictionaries, functions, loops, conditional statements, and console input/output in Python;
- to keep the project small enough that the program flow can be understood from the source file.

4. Functional Requirements

Functional area	What the current program does
Student entry	Accepts roll number, name, and marks for three subjects and adds a dictionary to the in-memory list.
Grade / calculation	Calculates total and average and passes the average to the grading helper in the source code.
Record display	Displays every student currently stored in the list.
Search	Searches for a student by roll number and prints the record when found.
Performance summary	Compares student averages to find the highest scorer.

These functions form the main workflow visible to a user. The current code does not include update/delete operations or permanent storage, so those features are not claimed here.

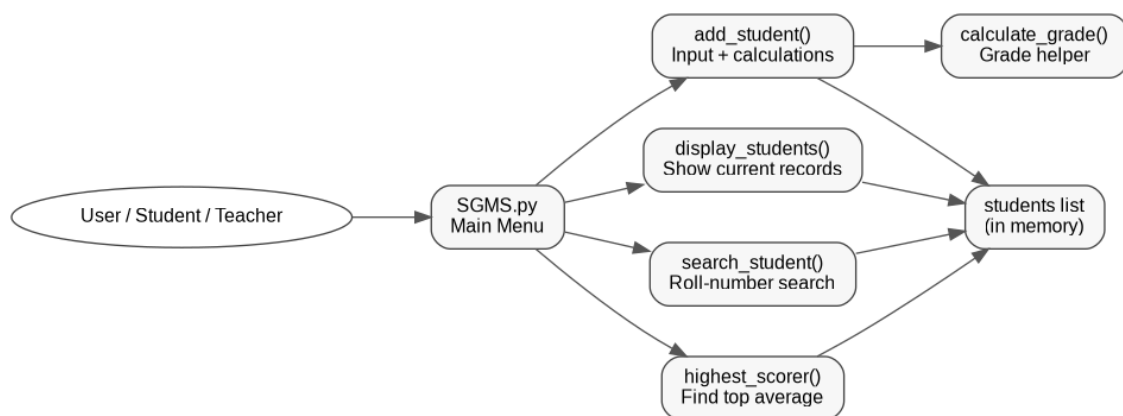
5. Non-Functional Requirements

Requirement	Current approach / status
Usability	A straightforward numbered menu and short prompts make the program easy to use from a terminal.
Reliability	For valid numeric input, the total, average, list storage, search, and highest-scorer comparison follow deterministic logic.
Performance	The program uses a simple Python list and loops, which is reasonable for the small in-memory dataset this project is intended for.
Maintainability	Different responsibilities are separated into named functions inside the single source file, so the flow can be followed function by function.
Error handling	Invalid menu choices are handled with an "Invalid" message. Numeric input is not wrapped in exception handling in the current source, so malformed numeric input can still stop the program.

6. System Architecture

The submitted implementation is a single-file procedural console application. `SGMS.py` contains the menu loop, the student list, the student-entry logic, the display logic, the search logic, the highest-scorer logic, and the grading helper. This architecture is intentionally small and does not require a database, web server, or external package.

Figure 1 - System Architecture



7. Design Diagrams

The following diagrams describe the program as it actually exists rather than adding features that are not present in the source.

Figure 2 - Use Case Diagram

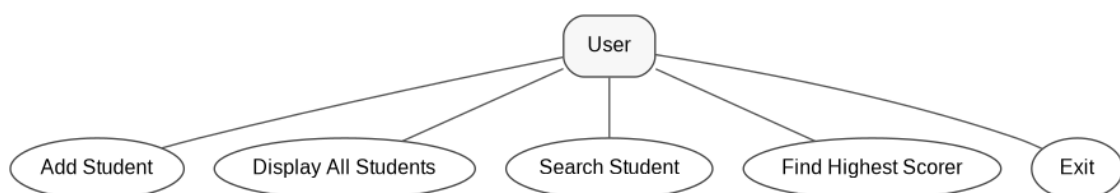


Figure 3 - Workflow Diagram

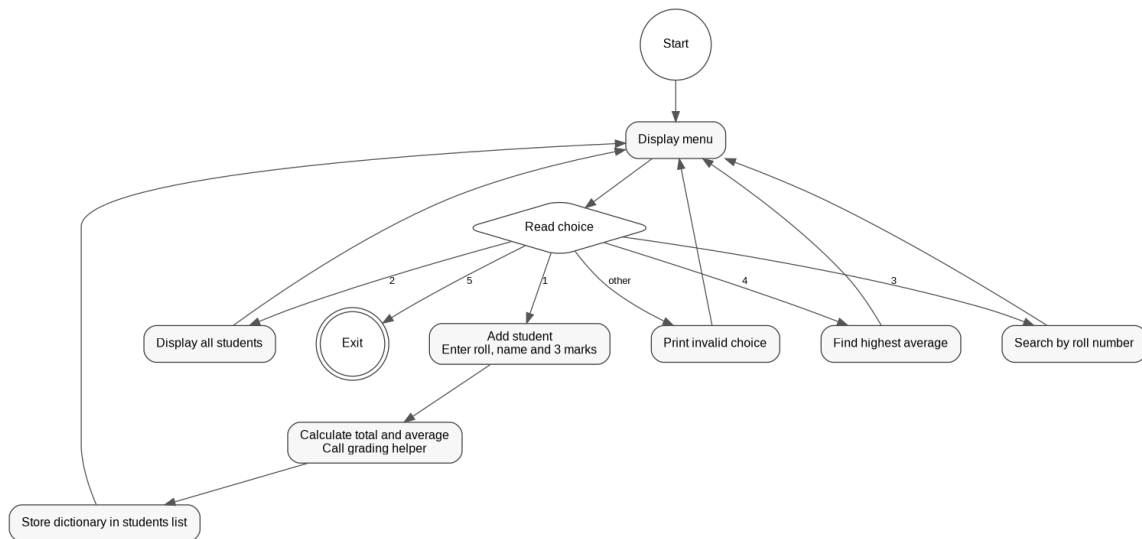


Figure 4 - Sequence Diagram

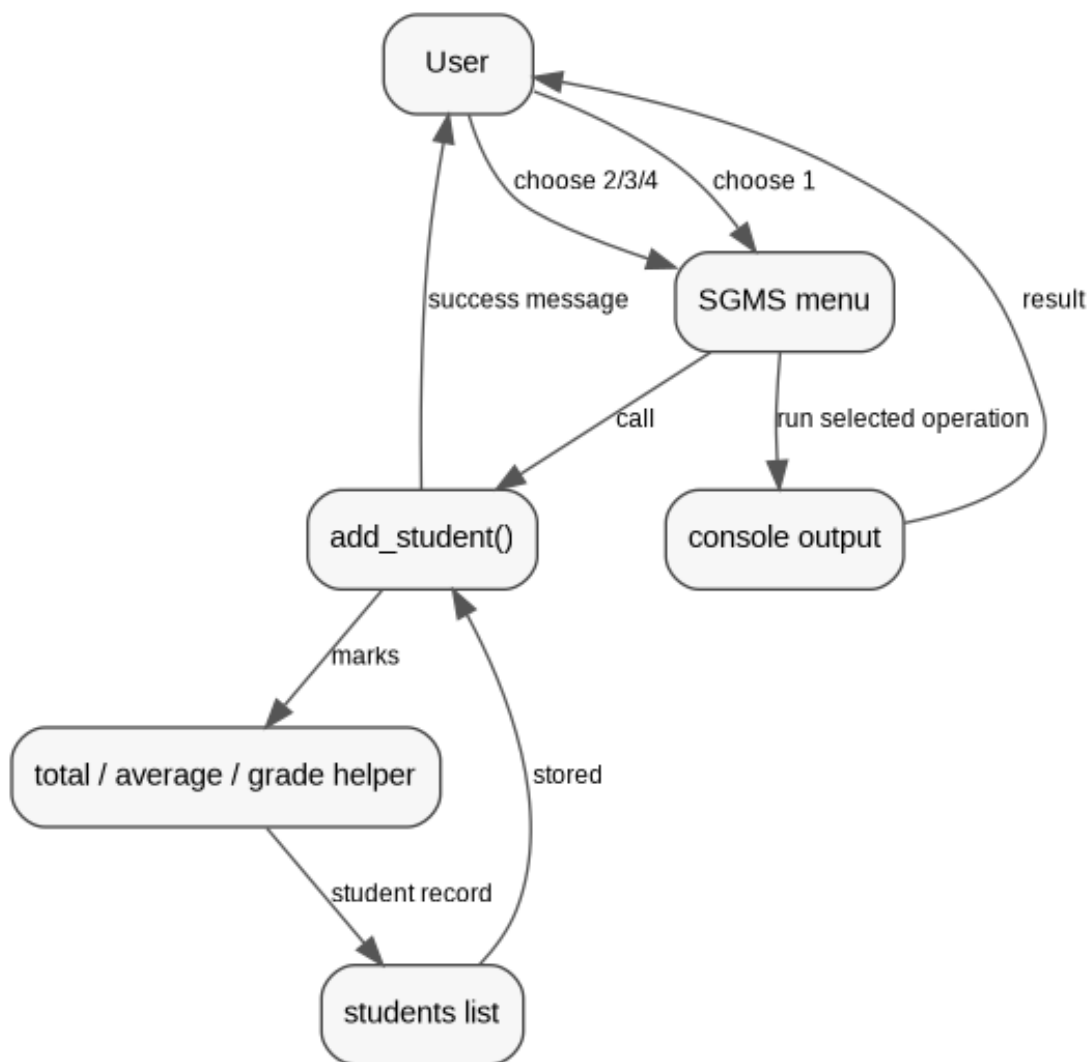


Figure 5 - Component / Function Diagram

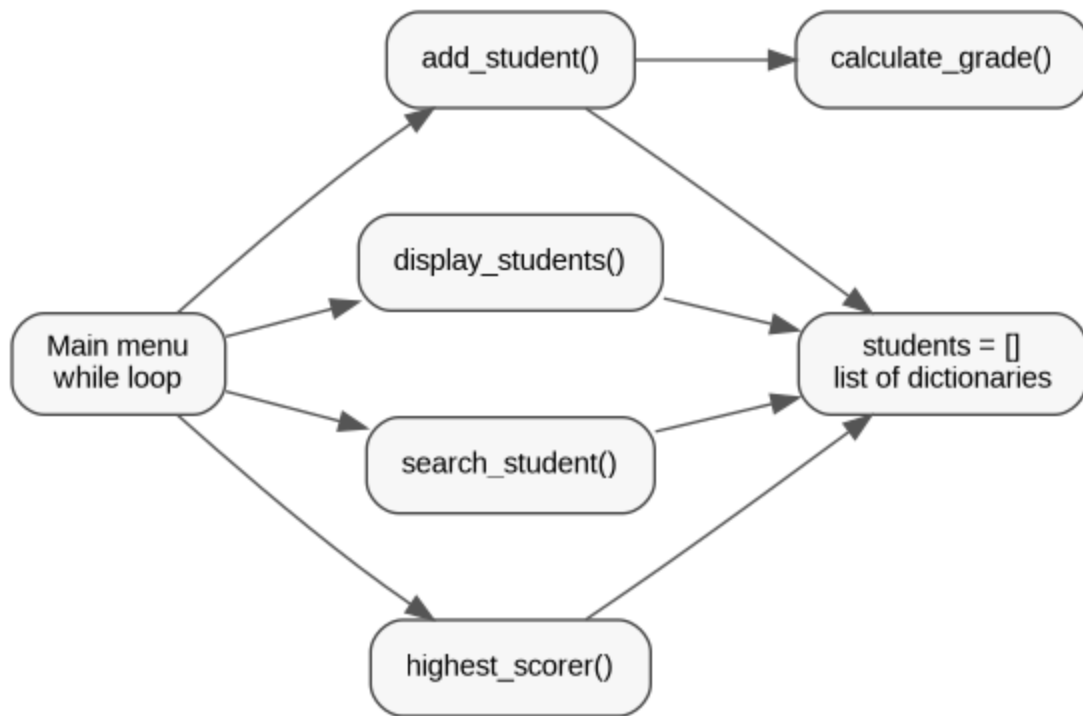
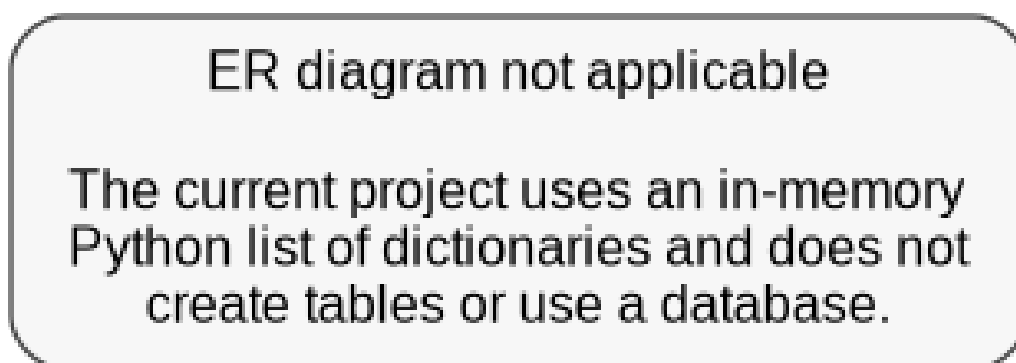


Figure 6 - ER Diagram / Storage Note



8. Design Decisions and Rationale

I chose a console interface because the subject is Python Programming and the main aim is to demonstrate programming fundamentals. A terminal menu keeps the focus on the program logic instead of UI development.

The program stores each student as a Python dictionary and keeps those dictionaries in a list called `students``. This is a natural fit for a small classroom example: the structure is easy to inspect, search, and loop through. It also avoids adding database setup to a project that does not require permanent storage.

The source uses separate functions for the major user actions. Even though everything remains in one file, the functions provide a clear separation between adding data, displaying data, searching, finding the highest scorer, and the grading helper.

No third-party packages are required for the program itself. This makes the project easy to run on a standard Python installation.

9. Implementation Details

Code element	Purpose
students = []	Stores student dictionaries in memory for the current program run.
calculate_grade()	Contains the visible grade thresholds inside a nested helper function.
add_student()	Reads inputs, calculates total and average, creates a dictionary, and appends it to the list.
display_students()	Loops through the list and prints each stored student.
search_student()	Loops through the list and compares roll numbers.
highest_scorer()	Compares averages and keeps the student with the highest value seen so far.
while True menu	Keeps the console application running until option 5 is selected.

A representative source fragment is shown below to make the implementation easy to connect with the report:

```
total = mark1 + mark2 + mark3
average = total / 3
grade = calculate_grade(average)
students.append(student)
```

The source code supplied for submission has deliberately been preserved. The report therefore describes the observed behaviour of that exact file.

10. Screenshots / Results

I ran the uploaded source file directly with sample inputs for two students. The program successfully accepted the records, calculated totals and averages, displayed the records, searched by roll number, and selected the highest average. The output also revealed the current grade-printing behaviour described below.

```

=====
STUDENT GRADE MANAGEMENT
=====
1. Add Student
2. Display All Students
3. Search Student
4. Find Highest Scorer
5. Exit
Enter your choice: Add Student
Enter roll number: Enter student name: Enter Python marks: Enter Mathematics marks: Enter Programming marks:
Total marks: 240.0
Average: 80.0
Grade: None
=====
STUDENT GRADE MANAGEMENT
=====
1. Add Student
2. Display All Students
3. Search Student
4. Find Highest Scorer
5. Exit
Enter your choice: Add Student
Enter roll number: Enter student name: Enter Python marks: Enter Mathematics marks: Enter Programming marks:
Total marks: 277.0
Average: 92.33333333333333
Grade: None
=====
STUDENT GRADE MANAGEMENT
=====
1. Add Student
2. Display All Students
3. Search Student
4. Find Highest Scorer
5. Exit
Enter your choice: All students
Roll Number: 101
Name: Rahul
Total Marks: 240.0
Average: 80.0
Grade: None
Roll Number: 102
Name: Priya
Total Marks: 277.0
Average: 92.33333333333333
Grade: None
=====
STUDENT GRADE MANAGEMENT
=====

```

Figure 7 - Sample console output from the exact submitted SGMS.py file.

Observed result: for a sample student with 90, 80 and 70 marks, the program printed total = 240.0 and average = 80.0. The printed grade was `None`. This is documented as-is because the project source has not been changed.

11. Testing Approach

Because the main program is interactive and runs its menu loop immediately, the additional test file uses black-box subprocess testing. Each test starts `SGMS.py`, supplies keyboard-style input, and checks text printed to the terminal. This keeps the submitted source untouched while still demonstrating repeatable checks.

```
python test_sgms.py
```

The test set covers starting and exiting, adding a student, displaying students, searching by roll number, and finding the highest scorer. The current source behaviour concerning the grade value is deliberately not hidden: the tests confirm that a grade line is printed, while the report records that its current value is `None`.

.....

Ran 5 tests in 2.999s

OK

12. Challenges Faced

The main challenge with this version is not the size of the code but its limited state handling. Records exist only while the program is running, so closing the program clears everything. Another practical issue is that numeric inputs are read directly with `int()` and `float()` without a surrounding `try/except` block.

A further observation from testing is that the grading helper currently defines the grade thresholds inside a nested function but does not return the result of that helper to the outer function. Because the instruction for this submission was to keep the original source unchanged, I have treated that as a documented limitation rather than silently editing the code.

13. Learnings and Key Takeaways

This project helped connect Python syntax with a small real-world style problem. The most useful part was seeing how a simple dictionary can represent a student record and how a list can hold multiple records.

I also practiced writing functions for separate tasks, using loops for searching and comparison, using conditional statements for menu decisions, accepting console input, and testing the program through repeatable scripted input. Another lesson was that documentation should match the actual source code; the final report intentionally avoids claiming database storage, CRUD update/delete, or other features that are not present in the submitted file.

14. Future Enhancements

The project can be expanded later without changing the basic idea. Possible next steps are permanent file or database storage, update and delete operations, stronger input validation, more subjects, printable report generation, a graphical interface, and user login. These are future ideas only and are not presented as current features.

15. References

1. VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines (provided assignment document).
2. Python 3 Documentation - official language documentation: <https://docs.python.org/3/>
3. Project repository: <https://github.com/devansh26bce10750/SGMS>